

```

In [2]: from rdflib import Graph, URIRef, Literal, Namespace
        from rdflib.namespace import RDFS

        # Define the oboInOwl namespace
        oboInOwl = Namespace("http://www.geneontology.org/formats/oboInOwl#")

        # Properties that should allow only string literals
        string_literal_properties = {
            oboInOwl.hasDbXref,
            RDFS.comment,
            RDFS.label,
        }

        # Load the original TTL file
        input_file = "VAE_results.ttl"
        output_file = "filtered_VAE.ttl"

        # Parse the TTL file
        g = Graph()
        g.parse(input_file, format="turtle")

        # New graph for filtered triples
        filtered_graph = Graph()
        filtered_graph.bind("oboInOwl", oboInOwl)
        filtered_graph.bind("rdfs", RDFS)

        # Filter triples
        for s, p, o in g:
            if p in string_literal_properties:
                if isinstance(o, Literal):
                    filtered_graph.add((s, p, o))
            else:
                if isinstance(o, URIRef):
                    filtered_graph.add((s, p, o))

        # Write the filtered graph back to TTL
        filtered_graph.serialize(destination=output_file, format="turtle")

        print(f"Filtered TTL written to '{output_file}'")

```

Filtered TTL written to 'filtered_VAE.ttl'

```

In [13]: from rdflib import Graph, Literal, URIRef
        import matplotlib.pyplot as plt
        import math

        # Load the filtered TTL file
        filtered_file = "filtered_GAN.ttl"
        g = Graph()
        g.parse(filtered_file, format="turtle")

        # Count IRIs and Literals per predicate
        property_counts = {}

        for _, predicate, obj in g:

```

```

pred_str = str(predicate)
if pred_str not in property_counts:
    property_counts[pred_str] = {"literals": 0, "iris": 0}

if isinstance(obj, Literal):
    property_counts[pred_str]["literals"] += 1
elif isinstance(obj, URIRef):
    property_counts[pred_str]["iris"] += 1

# Helper for custom labels inside the donut
def make_autopct(sizes, labels):
    def autopct(pct):
        total = sum(sizes)
        count = int(round(pct * total / 100.0))
        index = 0 if count == sizes[0] else 1 # match size
        label = labels[index]
        return f"{label}\n{pct:.1f}%"
    return autopct

# Plot configuration
num_properties = len(property_counts)
cols = 3
rows = math.ceil(num_properties / cols)

fig, axs = plt.subplots(rows, cols, figsize=(cols * 5, rows * 4))
axs = axs.flatten()

for i, (pred, counts) in enumerate(property_counts.items()):
    labels = ['Literals', 'IRIs']
    sizes = [counts['literals'], counts['iris']]
    colors = ['#66c2a5', '#fc8d62']

    axs[i].pie(
        sizes,
        labels=None,
        autopct=make_autopct(sizes, labels),
        startangle=90,
        colors=colors,
        wedgeprops=dict(width=0.2)
    )
    axs[i].set_title(f"{pred}", fontsize=9)
    axs[i].axis('equal')

# Hide unused subplots
for j in range(i + 1, len(axs)):
    fig.delaxes(axs[j])

plt.tight_layout()
plt.savefig("GAN_donut_charts_all_properties.png", dpi=300)
plt.close()

```

```

In [17]: from rdflib import Graph
        from collections import defaultdict, Counter

        # Load TTL file
        g = Graph()

```

```

g.parse("filtered_GAN.ttl", format="turtle")

# Step 1: Map (subject, predicate) -> set of values
prop_value_counts = defaultdict(lambda: defaultdict(set))

for s, p, o in g:
    prop_value_counts[p][s].add(o)

# Step 2: Build distribution: predicate -> (number of values -> count of sub
distribution = {}

for p, subj_objs in prop_value_counts.items():
    count_distribution = Counter(len(objs) for objs in subj_objs.values())
    distribution[p] = count_distribution

# Step 3: Display the distribution
for p, dist in distribution.items():
    print(f"\nProperty: {p}")
    for num_values, count in sorted(dist.items()):
        print(f"    {num_values} value(s): {count} subject(s)")

```

Property: <http://www.w3.org/2002/07/owl#annotatedProperty>

- 1 value(s): 3855 subject(s)
- 2 value(s): 348 subject(s)
- 3 value(s): 8 subject(s)

Property: <http://www.geneontology.org/formats/oboInOwl#hasDbXref>

- 1 value(s): 3936 subject(s)
- 2 value(s): 628 subject(s)
- 3 value(s): 90 subject(s)

Property: <http://www.geneontology.org/formats/oboInOwl#hasBroadSynonym>

- 1 value(s): 3223 subject(s)
- 2 value(s): 618 subject(s)
- 3 value(s): 39 subject(s)

Property: <http://www.w3.org/2002/07/owl#annotatedSource>

- 1 value(s): 3713 subject(s)
- 2 value(s): 352 subject(s)
- 3 value(s): 25 subject(s)

Property: <http://www.geneontology.org/formats/oboInOwl#hasRelatedSynonym>

- 1 value(s): 2885 subject(s)
- 2 value(s): 419 subject(s)
- 3 value(s): 22 subject(s)

Property: <http://www.w3.org/1999/02/22-rdf-syntax-ns#type>

- 1 value(s): 2126 subject(s)
- 2 value(s): 4829 subject(s)

Property: <http://www.w3.org/2000/01/rdf-schema#subClassOf>

- 1 value(s): 2905 subject(s)
- 2 value(s): 245 subject(s)

Property: <http://www.w3.org/2000/01/rdf-schema#comment>

- 1 value(s): 3912 subject(s)
- 2 value(s): 504 subject(s)

Property: <http://www.geneontology.org/formats/oboInOwl#hasNarrowSynonym>

- 1 value(s): 3312 subject(s)
- 2 value(s): 737 subject(s)
- 3 value(s): 66 subject(s)

Property: <http://www.geneontology.org/formats/oboInOwl#hasExactSynonym>

- 1 value(s): 3173 subject(s)
- 2 value(s): 542 subject(s)
- 3 value(s): 33 subject(s)

Property: <http://www.w3.org/2002/07/owl#someValuesFrom>

- 1 value(s): 3577 subject(s)
- 2 value(s): 440 subject(s)
- 3 value(s): 53 subject(s)
- 4 value(s): 2 subject(s)

Property: <http://www.w3.org/2000/01/rdf-schema#label>

- 1 value(s): 4046 subject(s)

Property: <http://www.w3.org/2002/07/owl#onProperty>

1 value(s): 3477 subject(s)

2 value(s): 353 subject(s)

3 value(s): 21 subject(s)

```
In [28]: import matplotlib.pyplot as plt
import seaborn as sns
from rdflib import Graph
from collections import defaultdict, Counter
import pandas as pd

sns.set(style="whitegrid")

# Load TTL file
g = Graph()
g.parse("filtered_VAE.ttl", format="turtle")

# Collect data
prop_value_counts = defaultdict(lambda: defaultdict(set))
for s, p, o in g:
    prop_value_counts[p][s].add(o)

distribution = {}
for p, subj_objs in prop_value_counts.items():
    count_distribution = Counter(len(objs) for objs in subj_objs.values())
    distribution[p] = count_distribution

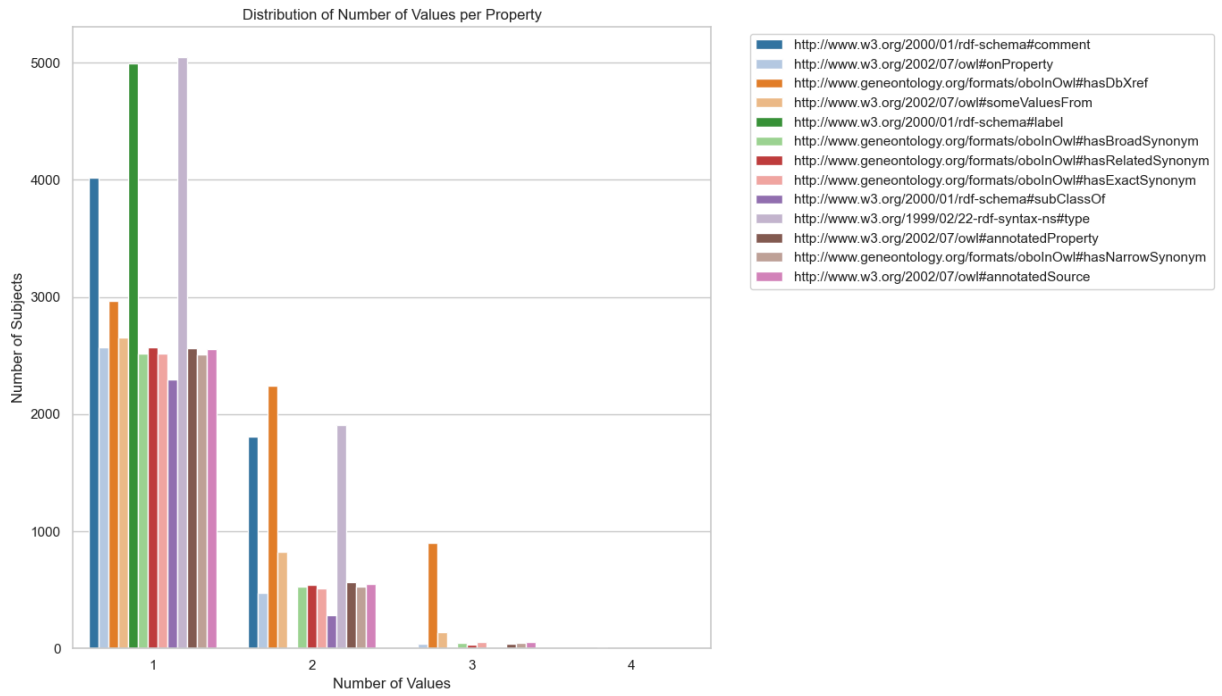
# Prepare data for seaborn
rows = []
for p, dist in distribution.items():
    for num_vals, count in dist.items():
        rows.append({
            "Property": str(p),
            "Number of Values": num_vals,
            "Number of Subjects": count,
        })

df = pd.DataFrame(rows)

# Plot with seaborn
plt.figure(figsize=(14, 8))
sns.barplot(
    data=df,
    x="Number of Values",
    y="Number of Subjects",
    hue="Property",
    palette="tab20"
)
plt.title("Distribution of Number of Values per Property")
plt.legend(bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()

# Save the figure as PNG file
plt.savefig("VAE_distribution_plot.png", dpi=300, bbox_inches='tight')

plt.show()
```



```
In [21]: import plotly.express as px
from rdflib import Graph
from collections import defaultdict, Counter
import pandas as pd

# Load TTL file
g = Graph()
g.parse("filtered_GAN.ttl", format="turtle")

# Collect data
prop_value_counts = defaultdict(lambda: defaultdict(set))
for s, p, o in g:
    prop_value_counts[p][s].add(o)

distribution = {}
for p, subj_objs in prop_value_counts.items():
    count_distribution = Counter(len(objs) for objs in subj_objs.values())
    distribution[p] = count_distribution

# Prepare dataframe
rows = []
for p, dist in distribution.items():
    for num_vals, count in dist.items():
        rows.append({
            "Property": str(p),
            "Number of Values": num_vals,
            "Number of Subjects": count,
        })

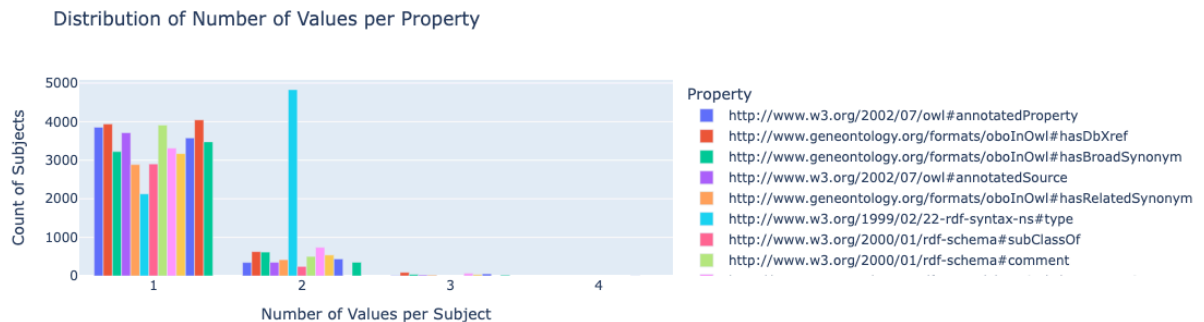
df = pd.DataFrame(rows)

# Interactive bar plot
fig = px.bar(
    df,
```

```

x="Number of Values",
y="Number of Subjects",
color="Property",
barmode='group',
title="Distribution of Number of Values per Property",
labels={"Number of Values": "Number of Values per Subject", "Number of S
hover_data=["Property"]
)
fig.show()

```



In [23]: `!pip install --upgrade kaleido`

```

Collecting kaleido
  Downloading kaleido-0.2.1-py2.py3-none-macosx_11_0_arm64.whl.metadata (15
kB)
  Downloading kaleido-0.2.1-py2.py3-none-macosx_11_0_arm64.whl (85.8 MB)
----- 85.8/85.8 MB 6.8 MB/s eta 0:00:0
000:0100:01
Installing collected packages: kaleido
Successfully installed kaleido-0.2.1

```

```

In [29]: from rdflib import Graph
from collections import defaultdict

input_file = "filtered_VAE.ttl"

g = Graph()
g.parse(input_file, format="turtle")

# Dictionary: property -> list of counts per subject
prop_subject_counts = defaultdict(list)

# For each property, we want to know for each subject how many objects it has
# So, we first create a nested dictionary: {property: {subject: count}}
prop_subj_objcount = defaultdict(lambda: defaultdict(int))

# Count objects per (subject, property)
for s, p, o in g:
    prop_subj_objcount[p][s] += 1

# For each property, get all counts per subject and store in a list
for p, subj_counts in prop_subj_objcount.items():
    counts = subj_counts.values()
    prop_subject_counts[p].extend(counts)

```

```
# Now calculate min and max per property
for p, counts in prop_subject_counts.items():
    print(f"{p}: min={min(counts)}, max={max(counts)}")
```

```
http://www.w3.org/2000/01/rdf-schema#comment: min=1, max=2
http://www.w3.org/2002/07/owl#onProperty: min=1, max=3
http://www.geneontology.org/formats/oboInOwl#hasDbXref: min=1, max=3
http://www.w3.org/2002/07/owl#someValuesFrom: min=1, max=4
http://www.w3.org/2000/01/rdf-schema#label: min=1, max=1
http://www.geneontology.org/formats/oboInOwl#hasBroadSynonym: min=1, max=3
http://www.geneontology.org/formats/oboInOwl#hasRelatedSynonym: min=1, max=3
http://www.geneontology.org/formats/oboInOwl#hasExactSynonym: min=1, max=3
http://www.w3.org/2000/01/rdf-schema#subClassOf: min=1, max=2
http://www.w3.org/1999/02/22-rdf-syntax-ns#type: min=1, max=2
http://www.w3.org/2002/07/owl#annotatedProperty: min=1, max=3
http://www.geneontology.org/formats/oboInOwl#hasNarrowSynonym: min=1, max=3
http://www.w3.org/2002/07/owl#annotatedSource: min=1, max=3
```

```
In [33]: from rdflib import Graph
        from collections import defaultdict

        input_file = "filtered_GAN.ttl"

        g = Graph()
        g.parse(input_file, format="turtle")

        # Nested dict: property -> subject -> count of objects
        prop_subj_objcount = defaultdict(lambda: defaultdict(int))

        # Count objects per (subject, predicate)
        for s, p, o in g:
            prop_subj_objcount[p][s] += 1

        # For each property, collect all distinct counts across subjects
        for p, subj_counts in prop_subj_objcount.items():
            counts = set(subj_counts.values()) # unique counts
            counts = sorted(counts)
            min_count = counts[0]
            max_count = counts[-1]

            # Get qname for nicer printing
            prefix, namespace, name = g.compute_qname(p)

            # Print summary including intermediate counts if any
            if len(counts) > 2:
                intermediate = counts[1:-1]
                print(f"{prefix}:{name}: min={min_count}, max={max_count}, intermedi
            elif len(counts) == 2:
                print(f"{prefix}:{name}: min={min_count}, max={max_count}")
            else:
                print(f"{prefix}:{name}: count={min_count}")
```

```

owl:annotatedProperty: min=1, max=3, intermediate=[2]
oboInOwl:hasDbXref: min=1, max=3, intermediate=[2]
oboInOwl:hasBroadSynonym: min=1, max=3, intermediate=[2]
owl:annotatedSource: min=1, max=3, intermediate=[2]
oboInOwl:hasRelatedSynonym: min=1, max=3, intermediate=[2]
rdf:type: min=1, max=2
rdfs:subClassOf: min=1, max=2
rdfs:comment: min=1, max=2
oboInOwl:hasNarrowSynonym: min=1, max=3, intermediate=[2]
oboInOwl:hasExactSynonym: min=1, max=3, intermediate=[2]
owl:someValuesFrom: min=1, max=4, intermediate=[2, 3]
rdfs:label: count=1
owl:onProperty: min=1, max=3, intermediate=[2]

```

In [32]: **import** matplotlib.pyplot **as** plt

```

data = {
    "rdfs:comment": {"min": 1, "max": 2, "intermediate": []},
    "owl:onProperty": {"min": 1, "max": 3, "intermediate": [2]},
    "oboInOwl:hasDbXref": {"min": 1, "max": 3, "intermediate": [2]},
    "owl:someValuesFrom": {"min": 1, "max": 4, "intermediate": [2, 3]},
    "rdfs:label": {"min": 1, "max": 1, "intermediate": []},
    "oboInOwl:hasBroadSynonym": {"min": 1, "max": 3, "intermediate": [2]},
    "oboInOwl:hasRelatedSynonym": {"min": 1, "max": 3, "intermediate": [2]},
    "oboInOwl:hasExactSynonym": {"min": 1, "max": 3, "intermediate": [2]},
    "rdfs:subClassOf": {"min": 1, "max": 2, "intermediate": []},
    "rdf:type": {"min": 1, "max": 2, "intermediate": []},
    "owl:annotatedProperty": {"min": 1, "max": 3, "intermediate": [2]},
    "oboInOwl:hasNarrowSynonym": {"min": 1, "max": 3, "intermediate": [2]},
    "owl:annotatedSource": {"min": 1, "max": 3, "intermediate": [2]},
}

properties = list(data.keys())

fig, ax = plt.subplots(figsize=(12, 6))

for i, p in enumerate(properties):
    ax.plot([i, i], [data[p]["min"], data[p]["max"]], color="gray", linewidth=2)
    ax.scatter(i, data[p]["min"], color="blue", label="min" if i == 0 else "")
    if data[p]["max"] != data[p]["min"]:
        ax.scatter(i, data[p]["max"], color="red", label="max" if i == 0 else "")
    for interm in data[p]["intermediate"]:
        ax.scatter(i, interm, color="green", label="intermediate" if i == 0 else "")

ax.set_xticks(range(len(properties)))
ax.set_xticklabels(properties, rotation=45, ha="right")
ax.set_ylabel("Number of values")
ax.set_title("Variance of number of values per property")
ax.legend()

plt.tight_layout()

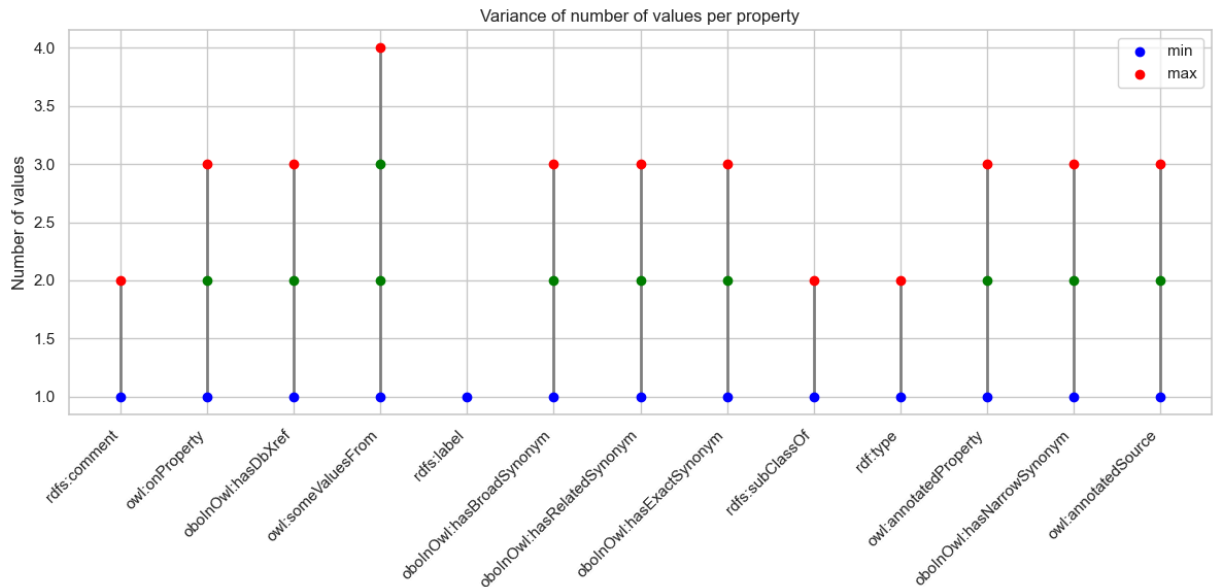
# Save figure as PNG image
output_path = "property_value_count_variance.png"
plt.savefig(output_path, dpi=300)

```

```
print(f"Plot saved as {output_path}")
```

```
plt.show()
```

Plot saved as property_value_count_variance.png



In [34]: `import matplotlib.pyplot as plt`

```
data = {
    "owl:annotatedProperty": {"min": 1, "max": 3, "intermediate": [2]},
    "oboInOwl:hasDbXref": {"min": 1, "max": 3, "intermediate": [2]},
    "oboInOwl:hasBroadSynonym": {"min": 1, "max": 3, "intermediate": [2]},
    "owl:annotatedSource": {"min": 1, "max": 3, "intermediate": [2]},
    "oboInOwl:hasRelatedSynonym": {"min": 1, "max": 3, "intermediate": [2]},
    "rdfs:type": {"min": 1, "max": 2, "intermediate": []},
    "rdfs:subClassOf": {"min": 1, "max": 2, "intermediate": []},
    "rdfs:comment": {"min": 1, "max": 2, "intermediate": []},
    "oboInOwl:hasNarrowSynonym": {"min": 1, "max": 3, "intermediate": [2]},
    "oboInOwl:hasExactSynonym": {"min": 1, "max": 3, "intermediate": [2]},
    "owl:someValuesFrom": {"min": 1, "max": 4, "intermediate": [2, 3]},
    "rdfs:label": {"count": 1}, # single count, no range
    "owl:onProperty": {"min": 1, "max": 3, "intermediate": [2]},
}
```

```
properties = list(data.keys())
```

```
fig, ax = plt.subplots(figsize=(14, 7))
```

```
for i, p in enumerate(properties):
    vals = data[p]
    # Single count property (no range)
    if "count" in vals:
        ax.scatter(i, vals["count"], color="purple", label="count=1" if i == 0 else None)
    else:
        ax.plot([i, i], [vals["min"], vals["max"]], color="gray", linewidth=1)
        ax.scatter(i, vals["min"], color="blue", label="min" if i == 0 else None)
        if vals["max"] != vals["min"]:
            ax.scatter(i, vals["max"], color="red", label="max" if i == 0 else None)
        for interm in vals["intermediate"]:
            ax.scatter(i, interm, color="green", label="intermediate" if i == 0 else None)
```

```

ax.scatter(i, interm, color="green", label="intermediate" if i =

ax.set_xticks(range(len(properties)))
ax.set_xticklabels(properties, rotation=45, ha="right")
ax.set_ylabel("Number of values")
ax.set_title("Variance of number of values per property")
ax.legend()

plt.tight_layout()

output_path = "property_value_count_variance_updated.png"
plt.savefig(output_path, dpi=300)
print(f"Plot saved as {output_path}")

plt.show()

```

Plot saved as property_value_count_variance_updated.png

